day07_RSYSLOG系统日志管理

今日内容：

- 1- 布置的DNS的综合案例
- 2- 日志基本介绍：
  - 【了解】基本介绍
  - 【实操】如何手动生成日志
- 3- RSYSLOG系统日志相关操作：
  - 【了解】3.1 常见的系统日志
  - 【理解+记录】3.2 日志服务：rsyslogd
  - 【实操】3.3 系统日志的相关综合
- 4- 日志轮替：
  - 【了解】什么是日志轮替
  - 【理解+记录】日志轮替相关配置文件
  - 【实操】综合案例

1. 日志基本介绍

日志（Log）是系统、应用程序或服务在**运行过程中**记录的一系列事件、操作、错误或其他重要信息的集合。它用于跟踪系统状态、诊断问题、分析性能，以及提供对系统活动的透明性。

> 日志就像是系统或程序的 "日记本"，会自动记录发生的事情，比如出错了、谁来访问了、做了什么操作等。这些记录会保存起来，方便我们事后查看，就像查监控录像一样。

日志的基本特点：

1. **记录时间戳**：每条日志都会标注发生时间。
2. **事件描述**：记录具体事件，比如错误详情、用户操作等。
3. **日志级别**：用级别标记事件的严重性（如普通信息、警告、严重错误）。

日志有什么用：

1. **系统维护**：帮管理员快速定位故障原因。
2. **安全审计**：追踪可疑操作（比如多次登录失败）。

3. **性能分析**：分析系统卡顿或资源占用问题。
4. **追踪记录**：记录关键操作（如数据库修改、文件变动）。

常见的日志类型：

1. **系统日志**：记录电脑/服务器的基础运行状态（比如用户登录、系统错误）。
2. **应用日志**：记录某个程序的运行情况（比如网站访问记录、程序报错）。
3. **安全日志**：专门记录安全问题（比如非法访问、权限变更）。
4. **自定义日志**：程序员按需求自己设计的记录（比如记录用户点击行为）。

常见日志管理工具:

1. **Linux 系统日志**：如Linux的`journalctl`（查系统日志）、`rsyslog`（存日志文件）。
2. **集中化日志管理**：如ELK（把多台设备的日志统一存储、分析、展示）。
3. **日志收集工具（数据采集工具）**：如Filebeat（定时收集日志并发送到指定位置）。

日志命令: logger

```
    logger 是一个用于向系统日志（syslog）写入日志消息的命令行工具。通过 logger，您可以生成日志记录，模拟各种日志级别，并将日志写入不同的日志文件，供系统管理员或程序查看和调试。它通常用于脚本或命令行中，来创建自定义日志消息。

格式:
    logger [OPTION] MESSAGE

    OPTION: 指定日志的各种选项，如日志级别、日志标识等。
    MESSAGE: 要记录的日志内容。

常用选项:
    -p ：指定日志的优先级(priority)（日志级别）。
    例如: -p local0.debug 表示将日志记录为 local0 设施，debug 严重性级别。

    -f: 将日志写入指定的文件(file)，而不是默认的系统日志文件。
    -i: 在日志中包含调用程序的进程ID（PID）。
    -s: 同时将日志消息输出到标准输出（终端）和系统日志。
    -n: remote_host: 将日志发送到远程主机的 syslog 守护进程。


示例:
    写入默认日志文件（/var/log/messages）:
```

```
    logger "This is a test log message"

    指定日志级别（如 crit）：
    logger -p crit "Critical error in system"
```

2. RSYSLOG系统日志

在 CentOS Stream 9 中日志服务已经由 rsyslogd 取代了早期的 syslogd 服务。Redhat 认为 syslogd 已经不能满足在工作中的需求，rsyslogd 相比 syslogd 具有一些新的特点：

- 基于 TCP 网络协议传输日志信息

```
传统的 syslogd 使用 UDP 协议发送日志数据，虽然效率高，但容易丢包。
rsyslogd 支持 TCP 协议，可以保证日志数据的可靠传输，特别适合重要信息的记录和远程日志传输。
```

- 更安全的网络传输方式

```
支持加密传输（如 TLS），即使在公共网络上传输日志，也能防止数据被窃听或篡改。
满足企业对日志传输安全性的要求，特别是敏感数据的记录。
```

- 有日志消息的及时分析框架

```
内置了日志分析和过滤的功能，能够对日志信息进行实时处理。
可快速筛选出关键事件，适用于大规模日志分析场景。
```

- 支持后台数据库

```
可以直接将日志存储到数据库中（如 MySQL、PostgreSQL），方便后续分析和管理。
提供了更好的日志归档和查询能力，相比于传统文件存储更灵活。
```

- 配置文件中可以写简单的逻辑判断

```
支持条件判断，管理员可以灵活定义日志的处理规则，比如按照日志来源、优先级、关键词等分流存储。
配置更强大，可以应对复杂的日志管理需求
```

- 与 syslog 配置文件相兼容

```
1   rsyslogd 继承了 syslog 的传统配置方法（如 /etc/rsyslog.conf 格式），因此从旧系统迁移到新系统时非常方便。
2   适应性强，兼容传统工具和方法。
```

2.1 系统中常见的日志文件

日志文件	说　　明
/var/log/cron	记录了系统定时任务相关的日志。
/var/log/cups/	记录打印CPU信息的日志
/var/log/dmesg	记录了系统在开机时内核自检的信息。也可以使用 dmesg 命令直接查看内核自检信息。
/var/log/btmp	记录错误登录的日志。这个文件是二进制文件，不能直接 vi 查看，而要使用 lastb 命令查看
/var/log/lastlog	记录系统中所有用户最后一次的登录时间的日志。这个文件也是二进制文件，不能直接 vi，而要使用 lastlog 命令查看。
/var/log/mailog	记录邮件信息。
/var/log/message	记录系统重要信息的日志。这个日志文件中会记录 Linux 系统的绝大多数重要信息，如果系统出现问题时，首先要检查的就应该是这个日志文件。
/var/log/secure	记录验证和授权方面的信息，只要涉及账户和密码的程序都会记录。比如说系统的登录，ssh 的登录，su 切换用户，sudo 授权，甚至添加用户和修改用户密码都会记录在这个日志文件中。
/var/log/wtmp	永久记录所有用户的登录、注销信息，同时记录系统的启动、重启、关机事件。同样这个文件也是一个二进制文件，不能直接 vi，而需要使用 last 命令来查看。
/var/run/utmp	记录当前已经登录的用户的信息。这个文件会随着用户的登录和注销而不断变化，只记录当前登录用户的信息。同样这个文件不能直接vi，而要使用 w，who，users 等命令来查询。

```
1   与用户登录相关的日志：
2         /var/log/btmp ：  记录用户登录错误的日志，需要使用lastb命令查看即可
3         /var/log/wtmp ：  记录所有用户登录、注销以及系统启动 重启 关机事件日志，需要使用last命令查看
```

```
        /var/run/utmp ：  记录当前已经登录的用户信息， 需要使用w、who、users查询日志
        /var/log/lastlog:  记录用户最后一次登录时间的日志， 需要使用lastlog 命令查看日志

大部分的日志， 都是通过 tail 命令查看日志文件
系统日志:  /var/log/message

安全日志:  /var/log/secure
```

除了系统默认的日志之外，采用 RPM 方式 或者 DNF方式安装的系统服务也会默认把日志记录在/var/log/目录中（源码包安装的服务日志是在源码包指定目录中）。不过这些日志不是由 rsyslogd 服务来记录和管理的，而是各个服务使用自己的日志管理文档来记录自身日志。

日志文件	说　　明
/var/log/httpd/	RPM 包安装的 apache 服务的默认日志目录
/var/log/mail/	RPM 包安装的邮件服务的额外日志目录
/var/log/samba/	RPM 包安装的 samba 服务的日志目录
/var/log/sssd/	守护进程安全服务目录

2.2 日志服务 rsyslogd

2.2.1 日志文件格式

只要是由日志服务 rsyslogd 记录的日志文件，他们的格式是一样的。基本日志格式包含以下四列：

- 事件产生的时间；
- 发生事件的服务器的主机名；
- 产生事件的服务名或程序名；
- 事件的具体信息。

例如： /var/log/secure 日志

```
[root@node3 log]# tail -100f /var/log/secure
Dec 19 08:59:31 node1 sshd[861]: Server listening on 0.0.0.0 port 22.
Dec 19 08:59:31 node1 sshd[861]: Server listening on :: port 22.
Dec 19 08:59:32 node1 polkitd[919]: Loading rules from directory /etc/polkit-1/rules.d
Dec 19 08:59:32 node1 polkitd[919]: Loading rules from directory /usr/share/polkit-1/rules.d
Dec 19 08:59:32 node1 polkitd[919]: Finished loading, compiling and executing 4 rules
Dec 19 08:59:32 node1 polkitd[919]: Acquired the name org.freedesktop.PolicyKit1 on the system bus
Dec 19 08:59:35 node1 sshd[1368]: Accepted password for root from 192.168.88.1 port 53230 ssh2
Dec 19 08:59:36 node1 systemd[1373]: pam_unix(systemd-user:session): session opened for user root(uid=0) by root(uid=0
Dec 19 08:59:36 node1 sshd[1368]: pam_unix(sshd:session): session opened for user root(uid=0) by root(uid=0)
Dec 19 08:59:36 node1 sshd[1370]: Accepted password for root from 192.168.88.1 port 53231 ssh2
Dec 19 08:59:36 node1 sshd[1370]: pam_unix(sshd:session): session opened for user root(uid=0) by root(uid=0)
Dec 19 09:00:35 node3 sshd[806]: Server listening on 0.0.0.0 port 22.
Dec 19 09:00:35 node3 sshd[806]: Server listening on :: port 22.
```

例如： /var/log/message日志

```
[root@node3 log]# tail -100f /var/log/messages
Dec 21 17:04:53 node3 systemd[1]: Reached target Sound Card.
Dec 21 17:04:53 node3 systemd[1]: Starting Authorization Manager...
Dec 21 17:04:53 node3 polkitd[946]: Started polkitd version 0.117
Dec 21 17:04:53 node3 systemd[1]: Started Authorization Manager.
Dec 21 17:04:53 node3 NetworkManager[860]: <info>  [1734771893.6962] device (ens160): s
Dec 21 17:04:53 node3 NetworkManager[860]: <info>  [1734771893.6972] policy: set 'ens16
Dec 21 17:04:53 node3 NetworkManager[860]: <info>  [1734771893.6989] device (ens160): s
Dec 21 17:04:53 node3 NetworkManager[860]: <info>  [1734771893.7006] device (ens160): s
Dec 21 17:04:53 node3 NetworkManager[860]: <info>  [1734771893.7009] device (ens160): s
Dec 21 17:04:53 node3 NetworkManager[860]: <info>  [1734771893.7016] manager: NetworkMa
Dec 21 17:04:53 node3 NetworkManager[860]: <info>  [1734771893.7039] device (ens160): A
Dec 21 17:04:53 node3 NetworkManager[860]: <info>  [1734771893.7043] manager: NetworkMa
Dec 21 17:04:53 node3 NetworkManager[860]: <info>  [1734771893.7045] manager: startup c
```

2.2.2 rsyslogd服务的配置文件

rsyslog的核心配置文件，位于：/etc/rsyslog.conf

```
[root@node3 log]# cat /etc/rsyslog.conf
# rsyslog configuration file

# For more information see /usr/share/doc/rsyslog-*/rsyslog_conf.html
# or latest version online at http://www.rsyslog.com/doc/rsyslog_conf.html
# If you experience problems, see http://www.rsyslog.com/doc/troubleshoot.html

#### GLOBAL DIRECTIVES ####

# Where to place auxiliary files
global(workDirectory="/var/lib/rsyslog")

# Use default timestamp format
module(load="builtin:omfile" Template="RSYSLOG_TraditionalFileFormat")

#### MODULES ####

module(load="imuxsock"    # provides support for local system logging (e.g. via logger command)
       SysSock.Use="off") # Turn off message reception via local log socket;
                          # local messages are retrieved through imjournal now.
module(load="imjournal"             # provides access to the systemd journal
       UsePid="system" # PID nummber is retrieved as the ID of the process the journal entry originates from
       FileCreateMode="0644" # Set the access permissions for the state file
       StateFile="imjournal.state") # File to store the position in the journal
#module(load="imklog") # reads kernel messages (the same are read from journald)
#module(load="immark") # provides --MARK-- message capability

# Include all config files in /etc/rsyslog.d/
include(file="/etc/rsyslog.d/*.conf" mode="optional")

# Provides UDP syslog reception
# for parameters see http://www.rsyslog.com/doc/imudp.html
#module(load="imudp") # needs to be done just once
```

详解：

```
1. 全局指令 (GLOBAL DIRECTIVES)

global(workDirectory="/var/lib/rsyslog")

作用：临时文件和运行数据（如队列、状态记录）都存这里，类似“工作文件夹”。

module(load="builtin:omfile" Template="RSYSLOG_TraditionalFileFormat")

作用：保存日志时，用最经典的格式（旧版样式），方便兼容老工具。

2. 模块 (MODULES)
# 使用不同的模块来提供各种功能。以下是一些常见模块的配置：
rsyslog

# 默认关闭本地套接字（比如logger命令的日志），改用下面的imjournal模块
imuxsock 模块
module(load="imuxsock" SysSock.Use="off")

# 直接从系统日志（systemd的journal）读取日志。
imjournal 模块
module(load="imjournal" UsePid="system" FileCreateMode="0644"
StateFile="imjournal.state")
# StateFile="imjournal.state"：记录读到哪了，下次接着读
# FileCreateMode="0644"：生成的文件权限是所有人可读


#注释掉的模块
# 读内核日志（现在用systemd代替了）。
#module(load="imklog") # reads kernel messages (the same are read from
journald)
# 定时打--MARK--日志标记（没开）
#module(load="immark") # provides --MARK-- message capability

# 包含额外配置文件
# 可以往/etc/rsyslog.d/目录加其他配置文件，灵活扩展规则
include(file="/etc/rsyslog.d/*.conf" mode="optional")


# 接收远程日志（Syslog Reception)模块 [默认没开]
rsyslog

# UDP 514端口 接收：速度快但可能丢消息（像广播，不保证对方收到）
#module(load="imudp") # needs to be done just once
#input(type="imudp" port="514")

# TCP 514端口 接收：更可靠，但占用资源多（类似打电话确认收到）。
```

```
#module(load="imtcp") # needs to be done just once
#input(type="imtcp" port="514")

3. 日志规则 (RULES)
# 日志规则部分定义了不同日志级别和类别的日志存储位置。

# 内核日志到控制台
# 作用: 将所有内核消息（`kern.*`）输出到控制台。由于太多信息会使屏幕杂乱，因此通常会注释掉。
#kern.* /dev/console

# 系统日志存储
# 所有普通日志（排除邮件、登录、定时任务）存这里
*.info;mail.none;authpriv.none;cron.none /var/log/messages

# 安全日志
# 登录、权限相关的日志（如SSH登录失败）存这里，权限严格。
authpriv.* /var/log/secure

# 邮件日志
# 邮件日志，用“异步写入”（先放内存，攒一波再写硬盘，避免卡住）。
mail.* -/var/log/maillog

# cron 日志
# 定时任务（cron）的日志，比如任务何时执行。
cron.* /var/log/cron

# 紧急消息
# 紧急日志（比如系统崩溃）弹窗给所有登录的用户。
*.emerg :omusrmsg:*

# 新闻错误日志
# 将 `uucp` 和 `news` 类别中 `crit`（严重）及以上级别的错误存储到 `/var/log/spooler` 文件。
uucp,news.crit /var/log/spooler

# 启动日志
# 开机启动过程的日志存这里。
local7.* /var/log/boot.log

4. 日志转发 (Forwarding) 示例
# 实现将系统日志统一全部转发到其他的服务器， 这在进行系统日志收集中非常的重要

# action(type="omfwd"
# queue.filename="fwdRule1"
# queue.maxdiskspace="1g"
# queue.saveonshutdown="on"
```

```
# queue.type="LinkedList"
# action.resumeRetryCount="-1"
# Target="remote_host" Port="XXX" Protocol="tcp")

# omfwd模块
# 把日志转发到另一台机器，比如：
    # Target="remote_host" Port="XXX"：目标的IP和端口。
    # Protocol="tcp"：用TCP发（更可靠）。
    # queue.maxdiskspace="1g"：如果网络断了，最多存1G日志在本地等着。
    # action.resumeRetryCount="-1"：无限重试，直到发出去为止。
```

- 1- /etc/rsyslog.conf 日志规则配置文件格式

```
authpriv.*                                              /var/log/secure
#服务名称[连接符号]日志等级                              日志记录位置
#认证相关服务.所有日志等级                               记录在/var/log/secure 日志中
```

  - 服务名称：

    首先需要确定 rsyslogd 服务可以识别哪些服务的日志，也可以理解为那些服务委托了 rsyslogd 服务来代为管理日志

服务名称	说　　明
auth（LOG_AUTH）	安全和认证相关消息（不推荐使用，建议使用 **authpriv** 代替）。
authpriv（LOG_AUTHPRIV）	私密的安全和认证相关消息，通常记录用户登录、身份验证等敏感信息。
cron（LOG_CRON）	系统定时任务（`cron` 和 `at`）产生的日志，记录任务的运行情况和结果。
daemon（LOG_DAEMON）	与系统中的各种后台守护进程相关的日志，例如 `NetworkManager`、`firewalld` 等服务。
ftp（LOG_FTP）	与 FTP 服务（如 vsftpd）相关的日志，记录文件传输的行为和状态。
kern（LOG_KERN）	由内核产生的日志（非用户进程的日志），通常记录硬件和系统级事件，例如驱动程序加载、设备问题等。
local0-local7（LOG_LOCAL0-7）	本地使用预留的日志类别，适用于用户自定义日志记录，允许灵活分配特定服务或应用的日志管理。

lpr（LOG_LPR）	打印系统的日志，记录打印任务的提交、完成和错误信息。
mail（LOG_MAIL）	邮件系统的日志，记录邮件的发送、接收、错误等事件，适用于 `Postfix`、`Sendmail` 等服务。
news（LOG_NEWS）	新闻服务器相关的日志，记录与新闻组（如 Usenet）相关的活动（现代系统中很少使用）。
syslog（LOG_SYSLOG）	系统日志守护进程产生的日志信息（尽管服务名称变为 rsyslogd，但配置中仍使用该类别）。
user（LOG_USER）	用户相关的日志，记录非系统或守护进程的普通用户活动日志。
uucp（LOG_UUCP）	UUCP（Unix-to-Unix Copy Program）子系统日志，主要用于记录早期数据传输协议相关的活动（现代很少使用）。

- 连接符号

```
1    “.”代表只要比后面的等级高的（包含该等级）日志都记录下来。比如：“cron.info” 代表 cron 服务产生的日志，只要日志等级大于等于 info 级别，就记录
2    “.=”代表只记录所需等级的日志，其他等级的都不记录。比如：“*.=emerg”代表任何日志服务产生的日志，只要等级是 emerg 等级就记录。这种用法及少见，了解就好
3    “.!”代表不等于，也就是除了该等级的日志外，其他等级的日志都记录。
```

- 日志等级
  - 日志级别越低， 日志就越详细， 越容易排查错误，但是弊端日志太多了，一般建议为INFO即可
  - 日志级别的排序（有低到高）：
    - **debug < info < notice < warning < err(分水岭) < cirt <alert < emerg**

等 级 名 称	说　　明
debug（LOG_DEBUG）	一般的调试信息说明
info（LOG_INFO）	基本的通知信息
notice（LOG_NOTICE）	普通信息，但是有一定的重要性
warning（LOG_WARNING）	警告信息，但是还不回影响到服务或系统的运行
err（LOG_ERR）	错误信息，一般达到 err 等级的信息已经可以影响到服务或系统的运行了。

crit（LOG_CRIT）	临界状况信息，比 err 等级还要严重
alert（LOG_ALERT）	警告状态信息，比 crit 还要严重。必须立即采取行动
emerg（LOG_EMERG）	奔溃等级信息，系统已经无法使用了
*	代表所有日志等级，比如：“authpriv.*”代表 authpriv 认证信息服务产生的日志，所有的日志等级都记录

日志等级这里还可以识别“none”，如果日志等级是 none，就说明忽略这个日志服务，该服务的所有日志都不再记录。

- 日志记录位置

  日志记录位置就是当前日志输出到哪个日志文件中保存，当然也可以把日志输出到打印机打印， 或者输出到远程日志服务器上（当然日志服务器要允许接收才行）

```
日志文件的绝对路径：【大多数情况】
    这是最常见的日志保存方法，如“/var/log/secure”就是保存系统验证和授权信息日志的。

系统设备文件：【相对较少】
    如“/dev/lp0”代表第一台打印机，如果日志保存位置是打印机设备的话，当有日志时就会在打印机打印

转发给远程主机：  【针对特殊的个别服务需要发送给其他服务器的情况】
    因为可以选择使用 TCP 协议和 UDP 协议传输日志信息，所以有两种发送格式。
    如使用“ @192.168.88.102:514 ”，就会把日志内容使用 UDP 协议发送到 192.168.88.102 的 UDP 514 端口上；
    如果使用“@@192.168.88.102:514”就会把日志内容使用 TCP 协议发送到 192.168.88.102 的 TCP 514 端口上，其中 514 是日志服务默认端口。当然只要 192.168.88.102 同意接收此日志，就可以把日志内容保存在日志服务器上。

忽略或丢弃日志：【相对较少】
    如果接受日志的对象是“~”，代表这个日志不会记录，而被直接丢弃。如“local3.* ~”代表忽略 local3 服务类型所有的日志都不记录。
```

练习： 把所有服务的“临界点”以上的错误都保存在/var/log/alert.log *日志中*

- 步骤一： 打开系统日志的配置文件

```
vi /etc/rsyslog.conf
```

- 步骤二： 在规则模块中， 配置需求要求的内容

```
#### RULES ####
# Log all kernel messages to the console.
# Logging much else clutters up the screen.
#kern.*                                                 /dev/console

# Log anything (except mail) of level info or higher.
# Don't log private authentication messages!
*.info;mail.none;authpriv.none;cron.none                /var/log/mess

# The authpriv file has restricted access.
authpriv.*                                              /var/log/secu

# Log all the mail messages in one place.
mail.*                                                  -/var/log/mai


# Log cron stuff
cron.*                                                  /var/log/cron

# Everybody gets emergency messages
```

RULES 模块范围内配置相关的规则

```
把所有服务的“临界点”以上的错误都保存在/var/log/alert.log 日志中

格式写法:
*.crit    /var/log/alert.log

# 注意:  该目标文件如果不存在，会自动进行创建
```

```
# Save boot messages also to boot.log
local7.*                                                /var/log/boot.log

# ALL SERVICE  CRIT  TO  /var/log/alert.log
*.crit                                                  /var/log/alert.log


# ### sample forwarding rule ###
#action(type="omfwd"
```

添加相关的配置

配置到此模块的上面 RULES模块的范围里面

- 步骤三： 重启rsyslogd服务

```
systemctl restart rsyslog

systemctl status rsyslog
```

如何测试呢?

思路： 通过logger 写一些属于crit 级别日志， 以及还可以写一写比这个级别更高或者更低的观察显现

正常的结果： 只有比crit 级别日志 大于等于的日志才可以被写入

```
生成日志的格式:
    logger -p 日志级别 "日志内容"

查看日志的命令:
    tail -Nf 日志文件

    N表示：从后往前查看多少行
    f:  表示实时监听日志最新的变化
```

2.3 系统日志综合案例一

需求：基于node1中的/var/log/下的messages和secure日志， 完成将日志发送到日志服务器（node3）， 进行统一保存，要求保存到node3服务器的/var/log/messages和/var/log/secure

思路:

```
第一步:  修改 node1的日志配置文件， 将对应写入到messages和secure日志对应规则配置， 将其目的地变更为发送到远端服务器方案 【建议选择：TCP方案】

第二步:  重启node1的系统日志服务


第三步:  修改 node3的日志配置文件， 将用于接收远程日志的模块打开【tcp模块】

第四步:  重启node3的系统日志服务

第五步:  在node3中开放端口号


第六步:
    测试message日志:  在node1中通过logger生成日志数据， 分别观察node1的系统日志和node3的系统日志， 哪个日志文件中会产生新的日志【正常:node3打印】

    测试安全日志:  通过MX连接下node1，然后在关闭， 观察在这个过程中， node1和node3的安全日志谁会打印新的日志【正常:node3打印】
```

2.3.1 在node1中实操

- 1- 打开node1的系统日志配置文件

```
vi /etc/rsyslog.conf
```

```
#### RULES ####

# Log all kernel messages to the console.
# Logging much else clutters up the screen.
#kern.*                                                 /dev/console

# Log anything (except mail) of level info or higher.
# Don't log private authentication messages!
*.info;mail.none;authpriv.none;cron.none                /var/log/messages

# The authpriv file has restricted access.
authpriv.*                                              /var/log/secure

# Log all the mail messages in one place.
mail.*                                                  -/var/log/maillog
```

- 2- 修改原本要写入到messages和secure日志的规则配置
  - 需求：将这个日志的目的地更改为node3【建议：TCP方案】

```
#### RULES ####

# Log all kernel messages to the console.
# Logging much else clutters up the screen.
#kern.*                                                 /dev/console

# Log anything (except mail) of level info or higher.
# Don't log private authentication messages!
#*.info;mail.none;authpriv.none;cron.none                /var/log/messages
*.info;mail.none;authpriv.none;cron.none                @@192.168.88.103:514
# The authpriv file has restricted access.
#authpriv.*                                              /var/log/secure
authpriv.*                                              @@192.168.88.103:514
```

- 3- 重启node1的rsyslog的日志服务

```
systemctl restart rsyslog

ststemctl status  rsyslog
```

```
[root@node1 ~]# systemctl restart rsyslog
[root@node1 ~]# systemctl status rsyslog
● rsyslog.service - System Logging Service
     Loaded: loaded (/usr/lib/systemd/system/rsyslog.service; enabled; preset: enabled)
     Active: active (running) since Sun 2025-03-23 15:27:01 CST; 4s ago
       Docs: man:rsyslogd(8)
             https://www.rsyslog.com/doc/
   Main PID: 120691 (rsyslogd)
      Tasks: 3 (limit: 22929)
     Memory: 3.2M
        CPU: 22ms
     CGroup: /system.slice/rsyslog.service
             └─120691 /usr/sbin/rsyslogd -n

3月 23 15:27:01 node1.itcast.cn systemd[1]: Starting System Logging Service...
3月 23 15:27:01 node1.itcast.cn systemd[1]: Started System Logging Service.
3月 23 15:27:01 node1.itcast.cn rsyslogd[120691]: [origin software="rsyslogd" swVersion="8.2310
3月 23 15:27:01 node1.itcast.cn rsyslogd[120691]: imjournal: journal files changed, reloading...
3月 23 15:27:01 node1.itcast.cn rsyslogd[120691]: cannot connect to 192.168.88.103:514: No route
3月 23 15:27:01 node1.itcast.cn rsyslogd[120691]: action 'action-0-builtin:omfwd' suspended (mod
3月 23 15:27:01 node1.itcast.cn rsyslogd[120691]: cannot connect to 192.168.88.103:514: No route
3月 23 15:27:01 node1.itcast.cn rsyslogd[120691]: action 'action-0-builtin:omfwd' suspended (mod
lines 1-20/20 (END)
```

发现在状态中，看到系统存在错误的值， connot connect to 192.168.88..... 原因是因为还没有配置node3的接收方案， 导致无法发送，当前能看到这个错误 恰恰说明在node1的配置是正确的

2.3.2 在node3中实操

- 1- 打开node3的rsyslog的服务配置文件

```
1  vi /etc/rsyslog.conf
```

```
#### MODULES ####

module(load="imuxsock"    # provides support for local system logging (e.g. via logger
       SysSock.Use="off") # Turn off message reception via local log socket;
                          # local messages are retrieved through imjournal now.
module(load="imjournal"             # provides access to the systemd journal
       UsePid="system" # PID nummber is retrieved as the ID of the process the journal
 from
       FileCreateMode="0644" # Set the access permissions for the state file
       StateFile="imjournal.state") # File to store the position in the journal
#module(load="imklog") # reads kernel messages (the same are read from journald)
#module(load="immark") # provides --MARK-- message capability

# Include all config files in /etc/rsyslog.d/
include(file="/etc/rsyslog.d/*.conf" mode="optional")

# Provides UDP syslog reception
# for parameters see http://www.rsyslog.com/doc/imudp.html
#module(load="imudp") # needs to be done just once
#input(type="imudp" port="514")    UDP接收

# Provides TCP syslog reception
# for parameters see http://www.rsyslog.com/doc/imtcp.html
#module(load="imtcp") # needs to be done just once
#input(type="imtcp" port="514")    TCP接收
```

- 2- 修改rsyslog的服务的配置文件， 打开tcp接收模块

```
#### MODULES ####

module(load="imuxsock"    # provides support for local system logging (
       SysSock.Use="off") # Turn off message reception via local log so
                          # local messages are retrieved through imjour
module(load="imjournal"             # provides access to the systemd jo
       UsePid="system" # PID nummber is retrieved as the ID of the proc
 from
       FileCreateMode="0644" # Set the access permissions for the state
       StateFile="imjournal.state") # File to store the position in the
#module(load="imklog") # reads kernel messages (the same are read from
#module(load="immark") # provides --MARK-- message capability

# Include all config files in /etc/rsyslog.d/
include(file="/etc/rsyslog.d/*.conf" mode="optional")

# Provides UDP syslog reception
# for parameters see http://www.rsyslog.com/doc/imudp.html
#module(load="imudp") # needs to be done just once
#input(type="imudp" port="514")

# Provides TCP syslog reception
# for parameters see http://www.rsyslog.com/doc/imtcp.html
module(load="imtcp") # needs to be done just once
input(type="imtcp" port="514")   将前面得# 删除即可
```

- 3- 重启node3的rsyslog的服务

```
systemctl restart rsyslog

ststemctl status  rsyslog
```

```
[root@node3 data]# systemctl restart rsyslog
[root@node3 data]# systemctl status rsyslog
● rsyslog.service - System Logging Service
     Loaded: loaded (/usr/lib/systemd/system/rsyslog.service; enabled; preset: enabled)
     Active: active (running) since Sun 2025-03-23 15:32:04 CST; 5s ago
       Docs: man:rsyslogd(8)
             https://www.rsyslog.com/doc/
   Main PID: 44346 (rsyslogd)
      Tasks: 8 (limit: 22929)
     Memory: 1.6M
        CPU: 21ms
     CGroup: /system.slice/rsyslog.service
             └─44346 /usr/sbin/rsyslogd -n

3月 23 15:32:04 node3.itcast.cn systemd[1]: Starting System Logging Service...
3月 23 15:32:04 node3.itcast.cn systemd[1]: Started System Logging Service.
3月 23 15:32:04 node3.itcast.cn rsyslogd[44346]: [origin software="rsyslogd" swVersion="8.2310
3月 23 15:32:04 node3.itcast.cn rsyslogd[44346]: imjournal: journal files changed, reloading..
lines 1-16/16 (END)
```

- 4- 由于开放了接收日志的模块，该模块会占用514端口用于接收日志，如果要想成功接收到日志，将必须在node3将514端口放行

```
firewall-cmd --add-port 514/tcp --permanent
firewall-cmd --reload
firewall-cmd --list-all
```

```
[root@node3 data]# firewall-cmd --add-port 514/tcp --permanent
firewall-cmd --reload
firewall-cmd --list-all
success
success
public (active)
  target: default
  icmp-block-inversion: no
  interfaces: ens160
  sources:
  services: cockpit dhcpv6-client ssh
  ports: 53/udp 53/tcp 514/tcp
  protocols:
  forward: yes
  masquerade: no
  forward-ports:
  source-ports:
  icmp-blocks:
  rich rules:
[root@node3 data]#
```

2.3.3 综合测试

先测试：messages日志文件， 正常的结果是node3的messages能收到node1的系统日志， 而node1无法接收到

- 第一步： 打开node1 和 node3的messages日志

node1

```
[root@node1 ~]# tail -f /var/log/messages
Mar 23 13:35:52 node1 dnf[88874]: CentOS Stream 9 - BaseOS                      794  B/s | 6.0 kB     00:07
Mar 23 13:35:53 node1 dnf[88874]: CentOS Stream 9 - AppStream                    16 kB/s | 6.1 kB     00:00
Mar 23 13:35:57 node1 dnf[88874]: CentOS Stream 9 - Extras packages             1.9 kB/s | 7.9 kB     00:04
Mar 23 13:35:57 node1 dnf[88874]: 元数据缓存已建立。
Mar 23 13:35:57 node1 systemd[1]: dnf-makecache.service: Deactivated successfully.
Mar 23 13:35:57 node1 systemd[1]: Finished dnf makecache.
Mar 23 15:00:45 node1 systemd[1]: Starting dnf makecache...
Mar 23 15:00:45 node1 dnf[118889]: 元数据缓存近期已刷新。
Mar 23 15:00:45 node1 systemd[1]: dnf-makecache.service: Deactivated successfully.
Mar 23 15:00:45 node1 systemd[1]: Finished dnf makecache.
```

node3

```
[root@node3 ~]# tail -f /var/log/messages
Mar 23 15:32:04 node3 systemd[1]: rsyslog.service: Deactivated successfully.
Mar 23 15:32:04 node3 systemd[1]: Stopped System Logging Service.
Mar 23 15:32:04 node3 systemd[1]: rsyslog.service: Consumed 1.478s CPU time.
Mar 23 15:32:04 node3 systemd[1]: Starting System Logging Service...
Mar 23 15:32:04 node3 systemd[1]: Started System Logging Service.
Mar 23 15:32:04 node3 rsyslogd[44346]: [origin software="rsyslogd" swVersion="8.2310.0-
346" x-info="https://www.rsyslog.com"] start
Mar 23 15:32:04 node3 rsyslogd[44346]: imjournal: journal files changed, reloading...
 try https://www.rsyslog.com/e/0 ]
Mar 23 15:33:37 node3 named[26152]: resolver priming query complete
Mar 23 15:34:18 node3 named[26152]: resolver priming query complete
Mar 23 15:36:19 node3 named[26152]: resolver priming query complete
```

- 第二步： 在node1通过logger命令生成日志

```
1   logger  "system  log  node1 ......"
```

```
7. 192.168.88.101 (root)   10. 192.168.88.101 (root)   4. 192.168.88.102 (root)   3. 192.168.88.103 (root)
[root@node1 ~]# tail -f /var/log/messages
Mar 23 13:35:52 node1 dnf[88874]: CentOS Stream 9 - BaseOS                    794  B/s | 6.0
Mar 23 13:35:53 node1 dnf[88874]: CentOS Stream 9 - AppStream                  16 kB/s | 6.1
Mar 23 13:35:57 node1 dnf[88874]: CentOS Stream 9 - Extras packages           1.9 kB/s | 7.9
Mar 23 13:35:57 node1 dnf[88874]: 元数据缓存已建立。
Mar 23 13:35:57 node1 systemd[1]: dnf-makecache.service: Deactivated successfully.
Mar 23 13:35:57 node1 systemd[1]: Finished dnf makecache.
Mar 23 15:00:45 node1 systemd[1]: Starting dnf makecache...
Mar 23 15:00:45 node1 dnf[118889]: 元数据缓存近期已刷新。
Mar 23 15:00:45 node1 systemd[1]: dnf-makecache.service: Deactivated successfully.
Mar 23 15:00:45 node1 systemd[1]: Finished dnf makecache.

            并未发现， 有日志产生
```

```
7. 192.168.88.101 (root)   10. 192.168.88.101 (root)   4. 192.168.88.102 (root)   3. 192.168.88.103 (root)
346" x-info="https://www.rsyslog.com"] start
Mar 23 15:32:04 node3 rsyslogd[44346]: imjournal: journal files changed, reloading...
 try https://www.rsyslog.com/e/0 ]
Mar 23 15:33:37 node3 named[26152]: resolver priming query complete
Mar 23 15:34:18 node3 named[26152]: resolver priming query complete
Mar 23 15:36:19 node3 named[26152]: resolver priming query complete


                                    看到了来自于node1的日志消息

Mar 23 15:38:18 node1 root[121957]: system  log  node1 ......
Mar 23 15:38:19 node1 rsyslogd[120691]: action 'action-0-builtin:omfwd' resumed (modul
) [v8.2310.0-4.el9 try https://www.rsyslog.com/e/2359 ]
Mar 23 15:38:51 node3 named[26152]: network unreachable resolving './NS/IN': 2001:500:
Mar 23 15:38:51 node3 named[26152]: FORMERR resolving './NS/IN': 192.5.5.241#53
```

同理， 也可以类似于监测安全日志， 测试方式雷同， 仅在生产日志的时候， 需要通过开关MX会话的方式来监测

2.4 系统日志综合案例二

恢复案例一的内容， 尝试操作案例二

需求说明： 为node1中被rsyslog所管理的日志， **全部转发**到node3日志服务器，同时在本地也要保留，请完成远程日志转发，并确保在网络不稳定或远程主机不可用时依然能正常保存和转发日志

思路：

```
需求： 将node1中全部日志统一转发到node3来接收， 同时要求保证在node1也要本地保存一份

# 本地保存， 本省rsyslog日志就会进行本地保存，依靠规则模块相关配置 【不需要关心】

# 思考： 如何完成全部日志统一转发操作呢？ 能想到解决方案的前提一定是对rsyslog的配置文件熟悉

解决方案： 日志转发（Forwarding）

```

2.4.1 在node1中实操

- 1- 打开node1的rsyslog的服务配置文件

```
vi /etc/rsyslog.conf
```

```
# ### sample forwarding rule ###
#action(type="omfwd"                    调整该段的相关配置
# # An on-disk queue is created for this action. If the remote host is
# # down, messages are spooled to disk and sent when it is up again.
#queue.filename="fwdRule1"       # unique name prefix for spool files
#queue.maxdiskspace="1g"         # 1gb space limit (use as much as possible)
#queue.saveonshutdown="on"       # save messages to disk on shutdown
#queue.type="LinkedList"         # run asynchronously
#action.resumeRetryCount="-1"    # infinite retries if host is down
# # Remote Logging (we use TCP for reliable delivery)
# # remote_host is: name/ip, e.g. 192.168.0.1, port optional e.g. 10514
#Target="remote_host" Port="XXX" Protocol="tcp")
```

- 2- 修改配置文件

```
# ### sample forwarding rule ###
action(type="omfwd"
# # An on-disk queue is created for this action. If the remote host is
# # down, messages are spooled to disk and sent when it is up again.
queue.filename="fwdRule1"       # unique name prefix for spool files
queue.maxdiskspace="1g"         # 1gb space limit (use as much as possible)
queue.saveonshutdown="on"       # save messages to disk on shutdown
queue.type="LinkedList"         # run asynchronously
action.resumeRetryCount="-1"    # infinite retries if host is down
# # Remote Logging (we use TCP for reliable delivery)
# # remote_host is: name/ip, e.g. 192.168.0.1, port optional e.g. 10514
Target="192.168.88.103" Port="514" Protocol="tcp" )
```

1- 打开前面得#号

2

3

4

- 3- 重启rsyslog服务

```
systemctl restart rsyslog
ststemctl status rsyslog
```

```
[root@node1 ~]# vi /etc/rsyslog.conf
[root@node1 ~]# systemctl restart rsyslog
ststemctl status rsyslog
-bash: ststemctl: 未找到命令
[root@node1 ~]# systemctl status rsyslog
● rsyslog.service - System Logging Service
     Loaded: loaded (/usr/lib/systemd/system/rsyslog.service; enabled; preset: enabled)
     Active: active (running) since Sun 2025-03-23 16:30:12 CST; 6s ago
       Docs: man:rsyslogd(8)
             https://www.rsyslog.com/doc/
   Main PID: 139609 (rsyslogd)
      Tasks: 4 (limit: 22929)
     Memory: 5.3M
        CPU: 31ms
     CGroup: /system.slice/rsyslog.service
             └─139609 /usr/sbin/rsyslogd -n

3月 23 16:30:12 node1.itcast.cn systemd[1]: Starting System Logging Service...
3月 23 16:30:12 node1.itcast.cn rsyslogd[139609]: [origin software="rsyslogd" swVersion="8.2310.0-4.el9" x-pid="139609"
3月 23 16:30:12 node1.itcast.cn systemd[1]: Started System Logging Service.
3月 23 16:30:12 node1.itcast.cn rsyslogd[139609]: imjournal: journal files changed, reloading...  [v8.2310.0-4.el9 try
3月 23 16:30:12 node1.itcast.cn rsyslogd[139609]: cannot connect to 192.168.88.103:514: Connection refused [v8.2310.0-4
3月 23 16:30:12 node1.itcast.cn rsyslogd[139609]: action 'action-8-builtin:omfwd' suspended (module 'builtin:omfwd'), r
3月 23 16:30:12 node1.itcast.cn rsyslogd[139609]: cannot connect to 192.168.88.103:514: Connection refused [v8.2310.0-4
lines 1-19/19 (END)
```

发现在状态中，看到系统存在错误的值， connot connect to 192.168.88..... 原因是因为还没有配置node3的接收方案， 导致无法发送，当前能看到这个错误 恰恰说明在node1的配置是正确的

2.4.2 在node3中实操

- 1- 打开node3的rsyslog的服务配置文件

```
vi /etc/rsyslog.conf
```

```
#### MODULES ####

module(load="imuxsock"    # provides support for local system logging (e.g. via logger
       SysSock.Use="off") # Turn off message reception via local log socket;
                          # local messages are retrieved through imjournal now.
module(load="imjournal"             # provides access to the systemd journal
       UsePid="system" # PID nummber is retrieved as the ID of the process the journal
 from
       FileCreateMode="0644" # Set the access permissions for the state file
       StateFile="imjournal.state") # File to store the position in the journal
#module(load="imklog") # reads kernel messages (the same are read from journald)
#module(load="immark") # provides --MARK-- message capability

# Include all config files in /etc/rsyslog.d/
include(file="/etc/rsyslog.d/*.conf" mode="optional")

# Provides UDP syslog reception
# for parameters see http://www.rsyslog.com/doc/imudp.html
#module(load="imudp") # needs to be done just once
#input(type="imudp" port="514")    UDP接收

# Provides TCP syslog reception
# for parameters see http://www.rsyslog.com/doc/imtcp.html
#module(load="imtcp") # needs to be done just once
#input(type="imtcp" port="514")    TCP接收
```

- 2- 修改rsyslog的服务的配置文件， 打开tcp接收模块

```
#### MODULES ####

module(load="imuxsock"    # provides support for local system logging (
       SysSock.Use="off") # Turn off message reception via local log so
                          # local messages are retrieved through imjour
module(load="imjournal"             # provides access to the systemd jo
       UsePid="system" # PID nummber is retrieved as the ID of the proc
 from
       FileCreateMode="0644" # Set the access permissions for the state
       StateFile="imjournal.state") # File to store the position in the
#module(load="imklog") # reads kernel messages (the same are read from
#module(load="immark") # provides --MARK-- message capability

# Include all config files in /etc/rsyslog.d/
include(file="/etc/rsyslog.d/*.conf" mode="optional")

# Provides UDP syslog reception
# for parameters see http://www.rsyslog.com/doc/imudp.html
#module(load="imudp") # needs to be done just once
#input(type="imudp" port="514")

# Provides TCP syslog reception
# for parameters see http://www.rsyslog.com/doc/imtcp.html
module(load="imtcp") # needs to be done just once
input(type="imtcp" port="514")    将前面得# 删除即可
```

- 3- 重启node3的rsyslog的服务

```
systemctl restart rsyslog

ststemctl status  rsyslog
```

```
[root@node3 data]# systemctl restart rsyslog
[root@node3 data]# systemctl status rsyslog
● rsyslog.service - System Logging Service
     Loaded: loaded (/usr/lib/systemd/system/rsyslog.service; enabled; preset: enabled)
     Active: active (running) since Sun 2025-03-23 15:32:04 CST; 5s ago
       Docs: man:rsyslogd(8)
             https://www.rsyslog.com/doc/
   Main PID: 44346 (rsyslogd)
      Tasks: 8 (limit: 22929)
     Memory: 1.6M
        CPU: 21ms
     CGroup: /system.slice/rsyslog.service
             └─44346 /usr/sbin/rsyslogd -n

3月 23 15:32:04 node3.itcast.cn systemd[1]: Starting System Logging Service...
3月 23 15:32:04 node3.itcast.cn systemd[1]: Started System Logging Service.
3月 23 15:32:04 node3.itcast.cn rsyslogd[44346]: [origin software="rsyslogd" swVersion="8.2310
3月 23 15:32:04 node3.itcast.cn rsyslogd[44346]: imjournal: journal files changed, reloading..
lines 1-16/16 (END)
```

- 4- 由于开放了接收日志的模块，该模块会占用514端口用于接收日志，如果要想成功接收到日志，将必须在node3将514端口放行

```
firewall-cmd --add-port 514/tcp --permanent
firewall-cmd --reload
firewall-cmd --list-all
```

```
[root@node3 data]# firewall-cmd --add-port 514/tcp --permanent
firewall-cmd --reload
firewall-cmd --list-all
success
success
public (active)
  target: default
  icmp-block-inversion: no
  interfaces: ens160
  sources:
  services: cockpit dhcpv6-client ssh
  ports: 53/udp 53/tcp 514/tcp
  protocols:
  forward: yes
  masquerade: no
  forward-ports:
  source-ports:
  icmp-blocks:
  rich rules:
[root@node3 data]#
```

2.4.3 综合测试

以测试messages日志文件为例： 正常的结果是node1和node3的messages都能收到node1的系统日志

- 第一步： 打开node1 和 node3的messages日志

node1

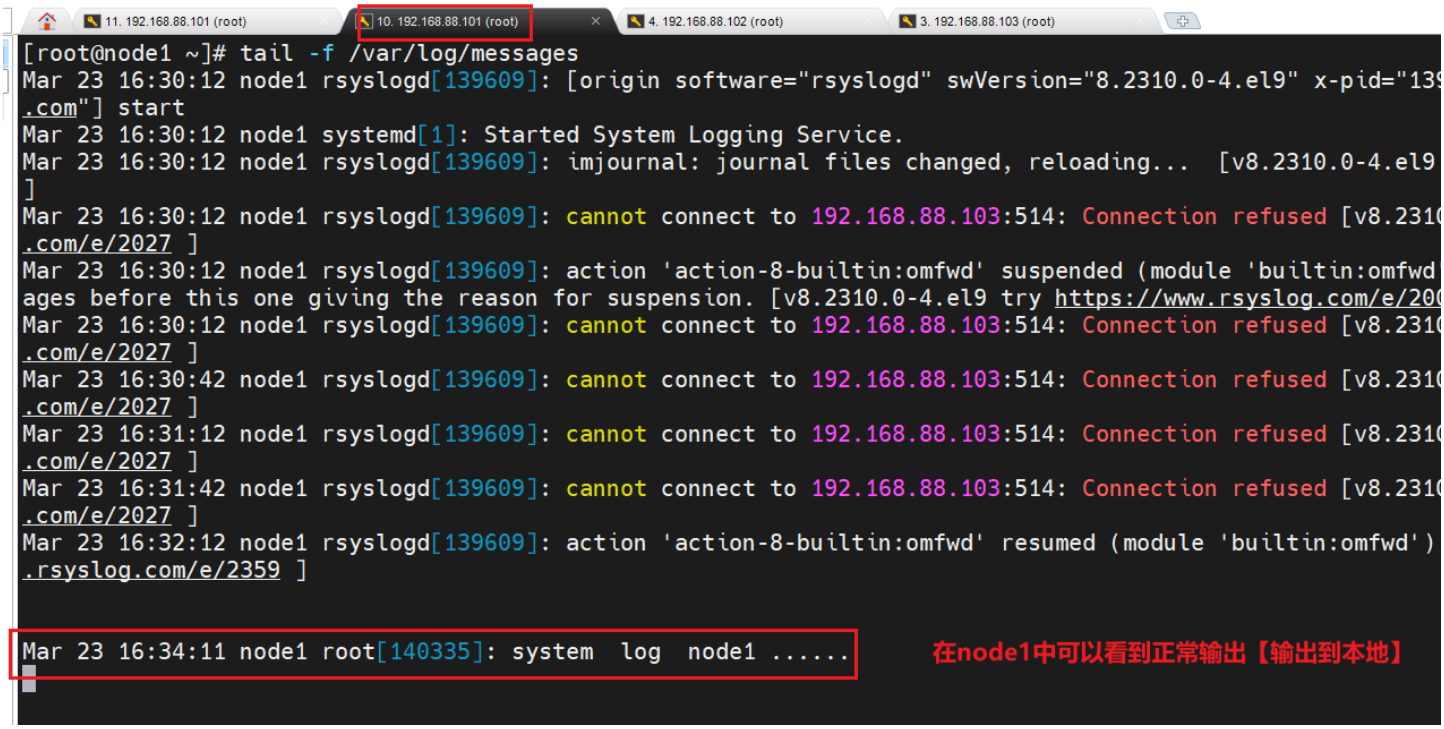
```
[root@node1 ~]# tail -f /var/log/messages
Mar 23 13:35:52 node1 dnf[88874]: CentOS Stream 9 - BaseOS                      794  B/s | 6.0 kB     00:07
Mar 23 13:35:53 node1 dnf[88874]: CentOS Stream 9 - AppStream                    16 kB/s | 6.1 kB     00:00
Mar 23 13:35:57 node1 dnf[88874]: CentOS Stream 9 - Extras packages             1.9 kB/s | 7.9 kB     00:04
Mar 23 13:35:57 node1 dnf[88874]: 元数据缓存已建立。
Mar 23 13:35:57 node1 systemd[1]: dnf-makecache.service: Deactivated successfully.
Mar 23 13:35:57 node1 systemd[1]: Finished dnf makecache.
Mar 23 15:00:45 node1 systemd[1]: Starting dnf makecache...
Mar 23 15:00:45 node1 dnf[118889]: 元数据缓存近期已刷新。
Mar 23 15:00:45 node1 systemd[1]: dnf-makecache.service: Deactivated successfully.
Mar 23 15:00:45 node1 systemd[1]: Finished dnf makecache.
```

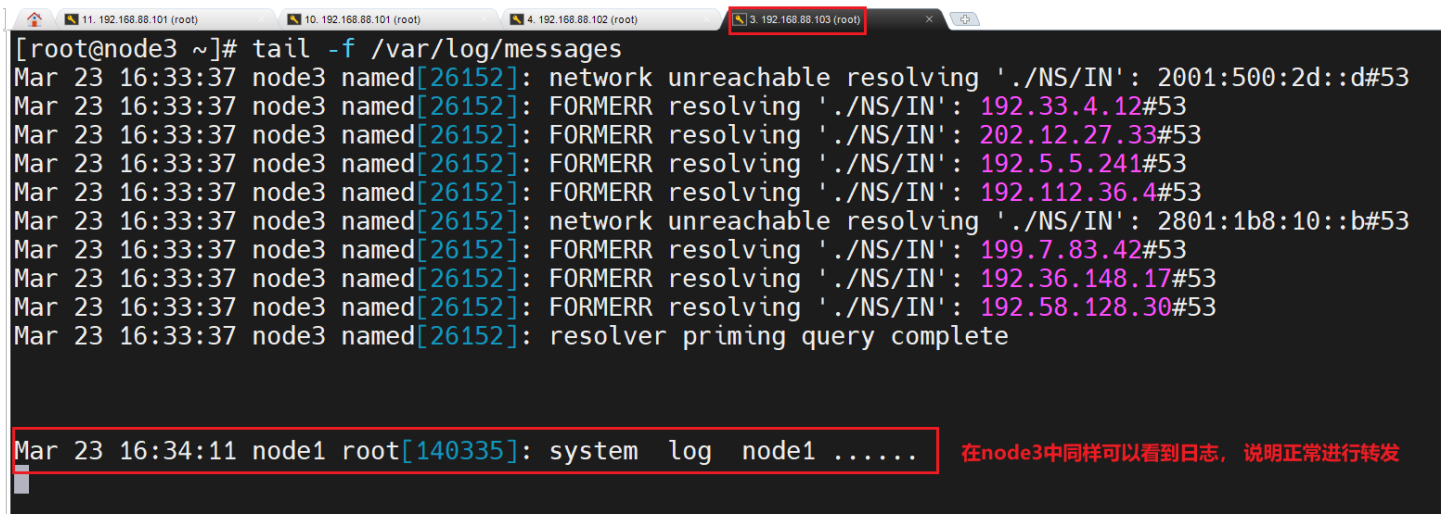
node3

```
[root@node3 ~]# tail -f /var/log/messages
Mar 23 15:32:04 node3 systemd[1]: rsyslog.service: Deactivated successfully.
Mar 23 15:32:04 node3 systemd[1]: Stopped System Logging Service.
Mar 23 15:32:04 node3 systemd[1]: rsyslog.service: Consumed 1.478s CPU time.
Mar 23 15:32:04 node3 systemd[1]: Starting System Logging Service...
Mar 23 15:32:04 node3 systemd[1]: Started System Logging Service.
Mar 23 15:32:04 node3 rsyslogd[44346]: [origin software="rsyslogd" swVersion="8.2310.0-
346" x-info="https://www.rsyslog.com"] start
Mar 23 15:32:04 node3 rsyslogd[44346]: imjournal: journal files changed, reloading...
 try https://www.rsyslog.com/e/0 ]
Mar 23 15:33:37 node3 named[26152]: resolver priming query complete
Mar 23 15:34:18 node3 named[26152]: resolver priming query complete
Mar 23 15:36:19 node3 named[26152]: resolver priming query complete
```

- 第二步： 在node1通过logger命令生成日志

```
logger "system log node1 ......"
```





同理， 也可以类似于监测安全日志， 测试方式雷同， 仅在生产日志的时候， 需要通过开关MX会话的方式来监测

小结：

```
通过刚刚演示的二个案例：
    案例一： 演示针对某一个服务 或者 某几个服务进行日志转发到其他服务器方案

    案例二： 演示针对整个系统日志统一转发到其他的服务器方案

实际场景价值呢?
    在进行集中化日志管理方案中， 我们就需要将机房中多个服务器的日志统一汇总到日志服务器中， 然后基于日志服务器进行统一的监控管理操作， 并最终部署一个告警和监控服务， 从而实现统一日志管理
```

```
8
9       对于系统日志服务而言：除了可以使用后续要学习专门用于采集日志的方案外， 我们也可以直接使用rsyslog所提供的这种日志转发的能力 从而更加方便完成日志转发到日志服务器的需求
10
11      其他价值： 能够对rsyslog的系统日志有一个基本认识， 大概清楚有哪些日志文件， 当后续系统发生相关问题的时候，我们可以通过对应的系统日志查询错误，进行解决问题
12
13      例如： 安全检查， 检查近期在这一周内，访问服务器超过100次的用户有哪些？ 看记录用户登录信息日志
```

3. 日志轮替

日志轮替是指对系统中生成的日志文件进行周期性管理的过程。随着时间的推移，日志文件会不断增长，最终可能占用大量磁盘空间。为了避免日志文件无限制增长并耗尽磁盘空间，日志轮替会定期将旧的日志文件归档或删除，创建新的日志文件。

日志轮替的主要目的是：

- 节省磁盘空间：定期清理过时的日志文件，避免占用过多磁盘空间。
- 保持日志的易管理性：通过轮替管理日志文件，确保日志的存储不会变得混乱。
- 提高性能：避免单个巨大日志文件影响系统性能，特别是在高流量环境下。

在 Linux 系统中，日志轮替通常由 logrotate 工具来执行。logrotate 会根据配置文件的规则定期轮替日志文件。

```
1   dnf install logrotate
2
3   默认安装了 logrotate 工具
```

logrotate 命令：

```
1   logrotate [选项] 配置文件名
2
3   选项：如果此命令没有选项，则会按照配置文件中的条件进行日志轮替
4       -v:         显示日志轮替过程。加了-v 选项，会显示日志的轮替的过程
5       -f:         强制进行日志轮替。不管日志轮替的条件是否已经符合，强制配置文件中所有的日志进行轮替
6
7   该命令主要是用于测试环节中使用， 用于测试配置的轮替方案是否生效
```

配置文件说明：

```
cat /etc/logrotate.conf

# 轮替周期：  指定日志轮替的周期为每周。可以设置为 daily、weekly、monthly 等。
weekly

# 保留日志文件的数量：保留 4 个日志文件的备份（即保留当前日志和 3 个旧日志）。
rotate 4

# 在日志轮替时，自动创建新的日志文件
create

# 使用日期作为日志轮替文件的后缀
dateext

# 在日志轮替时压缩旧日志。
#compress

# 包含/etc/logrotate.d/目录中所有的子配置文件。也就是说会把这个目录中所有子配置文件读取进来，进行日志轮替。
include /etc/logrotate.d



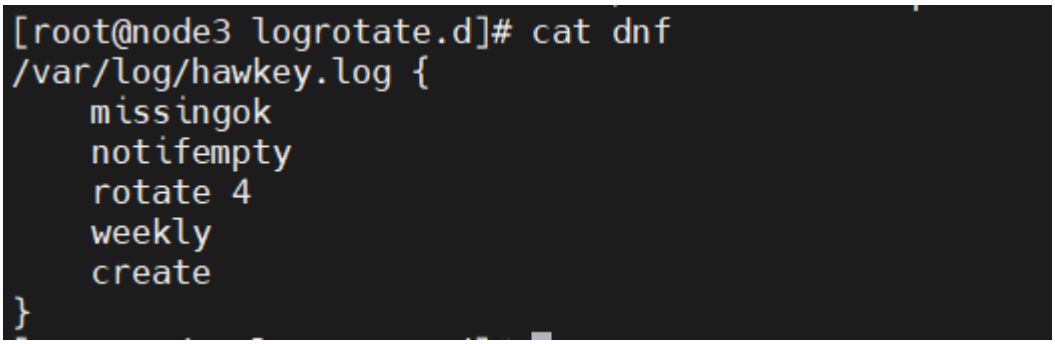
#以上日志日志配置为默认配置，如果需要轮替的日志没有设定独立的参数，那么都会遵守以上参数。
#如果轮替日志配置了独立参数，那么独立参数优先级更高。
```

如何为指定的日志设置轮替策略?

/etc/logrotate.d/ 目录中包含了各种服务的日志配置文件，每个文件指定了特定服务的日志轮替规则

```
[root@node3 logrotate.d]# pwd
/etc/logrotate.d
[root@node3 logrotate.d]# ll
总用量 32
-rw-r--r--. 1 root root 130 10月 14  2019 btmp
-rw-r--r--. 1 root root 160  8月 29 16:28 chrony
-rw-r--r--. 1 root root  88  9月  9  2022 dnf
-rw-r--r--. 1 root root  93  7月  3 00:56 firewalld
-rw-r--r--. 1 root root 162  9月 19 15:03 kvm_stat
-rw-r--r--. 1 root root 226  1月  8  2024 rsyslog
-rw-r--r--. 1 root root 289  7月 22 17:57 sssd
-rw-r--r--. 1 root root 145 10月 14  2019 wtmp
```

以dnf为例：




```
/var/log/hawkey.log {
    missingok    # 如果日志文件缺失，不报错。
    notifempty   # 如果日志文件为空，则不进行轮替。
    rotate 4     # 保留 4 个日志文件的备份（即保留当前日志和 3 个旧日志）。
    weekly       # 日志轮替的周期为每周
    create       # 创建新的日志文件
}
```

3.1 轮替相关配置参数

logrotate 配置文件的主要参数：

参　数	参　数　说　明
daily	日志的轮替周期是每天
weekly	日志的轮替周期是每周
monthly	日志的轮替周期是每月
rotate 数字	保留的日志文件的个数。0 指没有备份
compress	日志轮替时，旧的日志进行压缩
create [mode owner group]	建立新日志，同时指定新日志的权限与所有者和所属组。如create 0600 root utmp
mail address	当日志轮替时，输出内容通过邮件发送到指定的邮件地址。如mail shenc@lamp.net
missingok	如果日志不存在，则忽略该日志的警告信息
notifempty	如果日志为空文件，则不进行日志轮替
copytruncate	在进行日志轮替时，确保日志文件被轮替且不需要重启进程。作用是将原日志文件的内容复制到新的文件中，并且截断（清空）原日志文件，而不会影响进程的继续运行（存在丢失日志风险）。适合于低风险日志或无法中断进程的情况

minsize 大小	日志轮替的最小值。也就是日志一定要达到这个最小值才会轮替，否则就算时间达到也不轮替
size 大小	日志只有大于指定大小才进行日志轮替，而不是按照时间轮替。如 size 100k
dateext	使用日期作为日志轮替文件的后缀。如 secure-20180605
sharedscripts	在此关键字之后的脚本只执行一次
prerotate/endscript	在日志轮替之前执行脚本命令。endscript 标示 prerotate 脚本结束。
postrotate/endscript	在日志轮替之后执行脚本命令。endscript 标示 postrotate 脚本结束

这些参数中较为不好理解的应该就是 prerotate/endscript 和 postrotate/endscript 参数了， 我们利用“man logrotate”中的列子来解释下这两个参数。

```
/var/log/httpd/access.log
/var/log/httpd/error.log
{
    rotate 5 # 保留的日志文件的个数为5个
    mail recipient@example.org  # 信息发送到指定邮箱
    size 100k  # 日志大于 100KB 时才进行日志轮替，不再按照时间轮替
    sharedscripts # 无论有多少个日志文件需要轮替 以下脚本只执行一次
    postrotate # 在日志轮替结束之后，执行以下脚本
        /usr/bin/killall -HUP httpd # 重启 apache 服务
    endscript #脚本结束
}
```

prerotate 和 postrotate 主要用于在日志轮替的同时，执行指定的脚本，一般用于日志轮替之后重启服务。这里强调，如果你的日志是写入 rsyslog 服务的配置文件的，那么把新日志加入 logrotate 后，一定要重启 rsyslog 服务，否则你会发现虽然新日志建立了，但是数据还是写入了旧的日志当中。那是因为虽然 logrotate 知道日志轮替了，但是 rsyslog 服务却并不知道。同理，如果你的日志不是被 rsyslog 管理，如源码包安装的 Apache、Nginx 等服务，则需要重启 Apache 或 Nginx 服务，否则日志也不能正常轮替。

3.2 轮替案例

3.2.1 实战一：

实战1： /var/log/alert.log 该日志是我们通过/etc/rsyslog.conf 配置文件自己生成的日志，所以默认这个日志是不会轮替的

```
需求说明：
    1- 设置为周轮替一次
    2- 保留6个轮替日志
    3- 日志不存在，忽略日志的警告信息
    4- 在日志轮替之前执行去除日志a属性， 以便于能够顺利完成轮替操作
    5- 在日志轮替之后执行重启rsyslog服务：/bin/kill -HUP $(/bin/cat /var/run/rsyslogd.pid 2>/dev/null) &>/dev/null

注： 在配置前， 先对文件添加a属性，对日志文件进行保护
```

实操代码：

思路：

```
第一步： 先给 /var/log/alert.log  增加 a 属性
第二步： 配置轮替配置文件：
    思考：该文件放置的位置在哪里呢？ /etc/logrotate.d/

第三步：重启轮替的服务
    systemctl restart logrotate

第四步： 测试
    直接使用强制轮替命令， 强制触发轮替， 观察是否产生新的轮替文件
    logrotate -vf  /etc/logrotate.conf
    轮替完成后， 查看 /var/log/alert.log属性， 此时该属性的a属性应该是不存在的
```

- 1- 第一步： 先给 /var/log/alert.log 增加 a 属性

```
chattr +a /var/log/alert.log

lsattr /var/log/alert.log
```

```
[root@node1 log]# chattr +a /var/log/alert.log
[root@node1 log]# lsattr /var/log/alert.log
-----a---------------- /var/log/alert.log
```

- 2- 配置针对alert.log日志文件的轮替的配置文件

```
cd /etc/logrotate.d/
```

```
2
3   vi  alert
4
5   输入 i 插入以下内容:
6   /var/log/alert.log {
7           weekly
8           rotate 6
9           missingok
10          sharedscripts
11          prerotate
12                  chattr -a /var/log/alert.log
13          endscript
14          sharedscripts
15          postrotate
16                  /bin/kill -HUP $(/bin/cat /var/run/rsyslogd.pid 2>/dev/null)
    &>/dev/null
17          endscript
18  }
19
```

- 3- 重启logrotate.log

```
1   systemctl restart logrotate
2   systemctl status logrotate
```

```
[root@node1 logrotate.d]# systemctl restart logrotate
systemctl status logrotate
○ logrotate.service - Rotate log files
     Loaded: loaded (/usr/lib/systemd/system/logrotate.service; static)
     Active: inactive (dead) since Sun 2025-03-23 21:37:08 CST; 9ms ago
TriggeredBy: ● logrotate.timer
       Docs: man:logrotate(8)
             man:logrotate.conf(5)
    Process: 248162 ExecStart=/usr/sbin/logrotate /etc/logrotate.conf (code=exited, status=0/SUCCESS)
   Main PID: 248162 (code=exited, status=0/SUCCESS)
        CPU: 23ms

3月 23 21:37:08 node1.itcast.cn systemd[1]: Starting Rotate log files...
3月 23 21:37:08 node1.itcast.cn systemd[1]: logrotate.service: Deactivated successfully.
3月 23 21:37:08 node1.itcast.cn systemd[1]: Finished Rotate log files.
[root@node1 logrotate.d]#
```

可能重启的时候 会爆出如下的错误内容

```
[root@node1 logrotate.d]# systemctl restart logrotate
Job for logrotate.service failed because the control process exited with error code.
See "systemctl status logrotate.service" and "journalctl -xeu logrotate.service" for details.
[root@node1 logrotate.d]#
```

可以使用这两个命令 观察具体的错误

```
[root@node1 logrotate.d]# systemctl status logrotate
× logrotate.service - Rotate log files
     Loaded: loaded (/usr/lib/systemd/system/logrotate.service; static)
     Active: failed (Result: exit-code) since Sun 2025-03-23 21:38:04 CST; 54s ago
TriggeredBy: ● logrotate.timer
       Docs: man:logrotate(8)
             man:logrotate.conf(5)
    Process: 248526 ExecStart=/usr/sbin/logrotate /etc/logrotate.conf (code=exited, status=1/FAILURE)
   Main PID: 248526 (code=exited, status=1/FAILURE)
        CPU: 22ms

3月 23 21:38:04 node1.itcast.cn systemd[1]: Starting Rotate log files...
3月 23 21:38:04 node1.itcast.cn logrotate[248526]: error: alert:3 argument expected after rotate count
3月 23 21:38:04 node1.itcast.cn logrotate[248526]: error: found error in /var/log/alert.log , skipping
3月 23 21:38:04 node1.itcast.cn logrotate[248526]: error: found error in file alert, skipping
3月 23 21:38:04 node1.itcast.cn systemd[1]: logrotate.service: Main process exited, code=exited, status=1/FAILURE
3月 23 21:38:04 node1.itcast.cn systemd[1]: logrotate.service: Failed with result 'exit-code'.
3月 23 21:38:04 node1.itcast.cn systemd[1]: Failed to start Rotate log files.
[root@node1 logrotate.d]#
```

错误的描述信息

- 4- 执行强制轮替操作

```
1   logrotate -vf  /etc/logrotate.conf
```

执行完成后，测试操作：

- 测试一： 观察是否出现新的alter.log日志

```
[root@node1 logrotate.d]# cd /var/log/
[root@node1 log]# ll
总用量 1104
-rw-------  1 root   root        0  3月 23 21:44 alert.log
-rw-------  1 root   root      201  3月 23 21:27 alert.log-20250323
drwxr-xr-x. 2 root   root     4096  3月 12 18:50 anaconda
drwx------. 2 root   root       23  3月 12 18:55 audit
-rw-rw----  1 root   utmp        0  3月 23 21:44 btmp
-rw-rw----. 1 root   utmp        0  3月 12 18:46 btmp-20250323
drwxr-x---. 2 chrony chrony      6  9月 10  2024 chrony
-rw-------  1 root   root     4999  3月 23 21:01 cron
-rw-------. 1 root   root     2491  3月 21 11:18 cron-20250323
```

新日志文件

轮替后的文件

能看到这个， 说明 轮替成功了

- 测试二： 观察新的alert.log文件， 是否存在 a属性

```
[root@node1 log]# lsattr alert.log
---------------------- alert.log
```

如果能看没有任何属性， 也可以说明轮替成功了

【扩展补充1】

```
1   chattr命令：
2       一个用于修改 Linux 文件属性的命令。chattr 全称为 change attribute，它可以改变文件或目录的特殊属性，从而影响它们的行为，例如防止文件被修改、删除等。
3
```

```
格式：
    chattr [选项] [文件或目录]

常用选项：
    +：添加属性。
    -：移除属性。
    =：仅保留指定属性，移除其他所有属性

常见的文件属性：
    a：文件只能追加数据，不能被删除或修改现有内容（追加模式）。
    i：文件不可更改（不可删除、不可重命名、不可写入）。
    d：文件不会包含在备份工具（如 dump）中。
    s：文件被删除时，其数据会被安全地擦除（覆盖为零）。
    u：文件被删除时，其数据会被保留，可用于后续恢复。
    c：文件自动压缩，读取时解压缩，写入时压缩。
    t：禁止文件使用尾部合并（tail-merging，适用于某些文件系统）。
    e：文件具有扩展属性。

示例用法：
    1. 防止文件被删除或修改（设置不可更改属性）：
            chattr +i /etc/passwd
            添加 i 属性，使其不可更改。这意味着即使是 root 用户也无法删除、修改或重命名该文件，直到属性被移除。
    2. 设置文件为只能追加（日志文件保护）
            chattr +a /var/log/myapp.log
            设置为只能追加内容，任何覆盖或删除操作都会失败。这在保护日志文件时非常有用。

    3. 查看文件属性,文件的当前属性可以通过 lsattr 命令查看：
            lsattr /etc/passwd
    4. 移除属性：
            chattr -i /etc/passwd
    5. 批量操作,可以对整个目录及其内容设置属性：
            chattr -R +i /important_dir
```

【扩展补充2】

```
HUP 信号
    是 Linux/Unix 系统中的一种信号，完整名称为 Hangup Signal，信号编号是 1。它最初设计用于通知进程终端连接被挂断（如用户退出终端），但在现代系统中，它的用途已经扩展，特别是在管理守护进程和服务时被广泛使用。

作用：

    挂断通知：
```

```
        最初用途是向进程告知终端或连接已经挂断，例如用户通过电话调制解调器断开连接。
        进程收到 HUP 信号后，通常会终止。

    重新加载配置文件：
        现代守护进程收到 HUP 信号时，会重新加载配置文件，而不会终止进程。这是运维中常用的功能。
        守护进程在接收到 HUP 信号时会执行重新初始化操作，比如 Nginx、rsyslog、sshd 等服务。

    日志文件切换：
        某些进程（如 syslogd、rsyslogd）收到 HUP 信号后，会关闭当前日志文件并打开新的日志文件，常用于日志轮替。


注意： 不同的进程对应HUP信号的响应的方案是不一样的， 对应的应用是否支持哪种作用建议通过AI查询即可

一般使用kill命令向进程发送HUP信号：
格式：kill -HUP <PID>

示例一：
    kill -HUP 1234  向进程 ID 为 1234 的进程发送 HUP 信号。

示例二： 通过文件存储的 PID
    kill -HUP $(cat /var/run/nginx.pid)
    读取存储在 /var/run/nginx.pid 中的进程 ID，并向其发送 HUP 信号。

HUP 信号的具体行为由接收进程决定，经典的服务进程响应行为：
    sshd        重新加载配置文件，但不会中断现有会话。
    nginx        平滑地重新加载配置文件，不中断现有连接。
    rsyslog        切换日志文件或重新加载配置。
    bash        通知 Bash 终端断开连接。
    tmux        会话继续运行，但会分离挂断的终端。

并不是所有的应用都支持接收HUP信号，所以在使用HUP信号的时候，首先要先确定 目标进程是否支持HUP信号，以及支持具体行为是什么
```

3.2.2 实战二：

实战2： 针对tomcat日志完成轮替操作

```
先将tomcat要启动状态，启动OK后，在完操作

日志文件：   /usr/local/apache-tomcat-9.0.97/logs/catalina.out

需求说明：
```

```
1- 要求每日进行日志轮替
2- 保留7个日志备份文件
3- 压缩旧日志文件
4- 不能停止tomcat， 完成日志轮替操作
5- 如果日志不存在，忽略错误
6- 如果日志文件为空，则不轮替
```